МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №2

Специальность АС-67

Выполнил
В. М. Голуб,
студент группы АС-67

Проверил
А. А. Крощенко,
ст. преп. кафедры ИИТ,
«___> _____2026 г.

Брест 2026

Цель работы: изучить применение линейной и логистической регрессии для решения практических задач. Научиться обучать модели, оценивать их качество с помощью соответствующих метрик и интерпретировать результаты. **Вариант 1**

Вариант 1

- **Регрессия (Прогнозирование стоимости жилья в Калифорнии)**
  1. California Housing
  2. Предсказать медианную стоимость дома (median_house_value)
  3. **Задания:**
     - загрузите данные и разделите их на обучающую и тестовую выборки;
     - обучите модель **линейной регрессии** на обучающих данных;
     - сделайте предсказания для тестовой выборки;
     - оцените качество модели, рассчитав метрики **MSE** (Mean Squared Error) и **R^2** (Coefficient of Determination);
     - визуализируйте результат: постройте диаграмму рассеяния для признака median_income (медианный доход) и целевой переменной, нанеся на неё линию регрессии.
- **Классификация (Прогнозирование выживаемости на "Титанике")**
  1. Titanic
  2. Предсказать, выжил ли пассажир (Survived)
  3. **Задания:**
     - загрузите и предварительно обработайте данные (заполните пропуски, преобразуйте категории в числа);
     - обучите модель **логистической регрессии**;
     - оцените качество модели, рассчитав **Accuracy**, **Precision** и **Recall**;

- постройте и проанализируйте **матрицу ошибок** (confusion matrix).

Задание. выполнить задания по варианту (регрессия и классификация), построить все требуемые визуализации и рассчитать метрики.

Выполнение:

Код программы

```
import pandas as pd import numpy as np import matplotlib.pyplot as
plt from pathlib import Path from sklearn.model_selection import
train_test_split from sklearn.linear_model import LinearRegression,
LogisticRegression from sklearn.preprocessing import OneHotEncoder,
StandardScaler from sklearn.impute import SimpleImputer from
sklearn.compose import ColumnTransformer from sklearn.pipeline import
Pipeline from sklearn.metrics import (     mean_squared_error,
r2_score,     accuracy_score,     precision_score,     recall_score,
confusion_matrix,     ConfusionMatrixDisplay
)  folder =
Path(__file__).resolve().parent
 pd.set_option("display.max_columns",
None) pd.set_option("display.width", 1500)
 print("ЛИНЕЙНАЯ РЕГРЕССИЯ: CALIFORNIA
HOUSING")
 housing = pd.read_csv(folder /
"california_housing.csv")
 print("\nПервые 5 строк:")
print(housing.head().to_string(index=False))
 print("\nКоличество
пропусков:")
print(housing.isnull().sum())

X = housing.drop(columns=["median_house_value"]) y
= housing["median_house_value"]

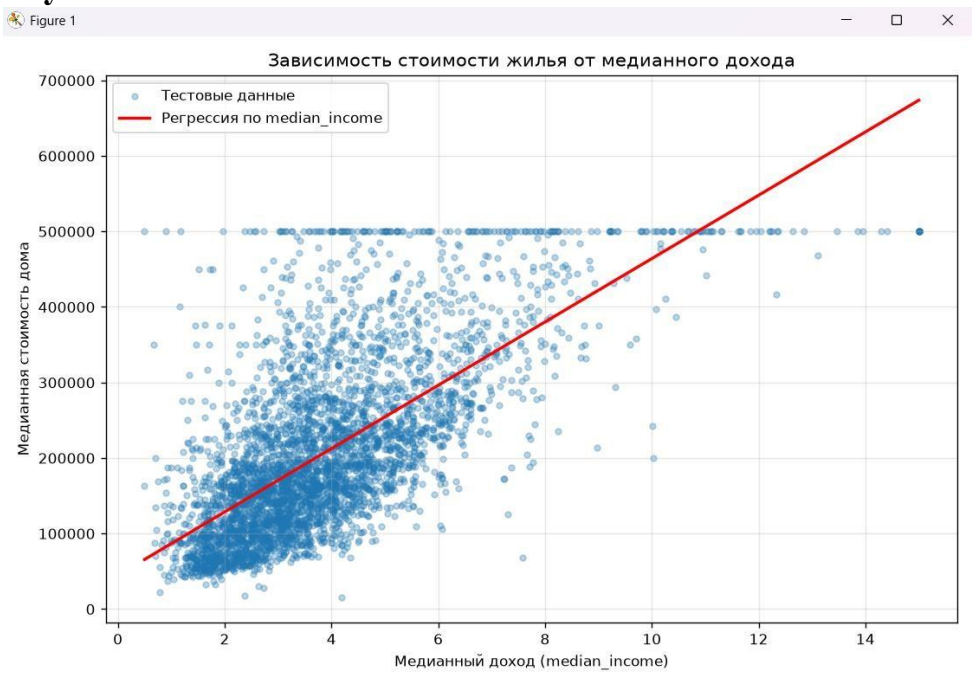
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)  numeric_columns = X.select_dtypes(include="number").columns.tolist()
categorical_columns = X.select_dtypes(exclude="number").columns.tolist()
 numeric_transformer =
Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])  categorical_transformer =
Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
   ("encoder", OneHotEncoder(handle_unknown="ignore"))
])  preprocessor =
ColumnTransformer([
    ("numeric", numeric_transformer, numeric_columns),
("categorical", categorical_transformer, categorical_columns)
])  linear_model = Pipeline([
("preprocessor", preprocessor),
    ("model", LinearRegression())
```

```
]) linear_model.fit(X_train,
y_train) y_pred =
linear_model.predict(X_test)
 print(f"\nОбучающая выборка: {len(X_train)}")
print(f"Тестовая выборка: {len(X_test)}") print(f"MSE:
{mean_squared_error(y_test, y_pred):.2f}") print(f"R²:
{r2_score(y_test, y_pred):.4f}")
 results =
pd.DataFrame({
    "Реальная стоимость": y_test.to_numpy(),
    "Прогноз": y_pred
}) print("\nПримеры предсказаний:")
print(results.head(10).round(2).to_string(index=False))
 income_model = LinearRegression()
income_model.fit(X_train[["median_income"]], y_train)
 income_values = pd.DataFrame({
"median_income": np.linspace(
        X_train["median_income"].min(),
        X_train["median_income"].max(),
        200
    )
}) price_values =
income_model.predict(income_values)
 plt.figure(figsize=(9,
6)) plt.scatter(
    X_test["median_income"],
y_test,     alpha=0.3,
s=15,     label="Тестовые
данные"
) plt.plot(
income_values["median_income"],
price_values,     color="red",
   linewidth=2,     label="Регрессия
по median_income"
) plt.title("Зависимость стоимости жилья от медианного
дохода") plt.xlabel("Медианный доход (median_income)")
plt.ylabel("Медианная стоимость дома") plt.legend()
plt.grid(alpha=0.3) plt.tight_layout()
 print("\n   ЛОГИСТИЧЕСКАЯ РЕГРЕССИЯ: TITANIC
")
 titanic = pd.read_csv(folder / "Titanic-
Dataset.csv")
 print("\nПервые 5 строк:")
print(titanic.head().to_string(index=False))
 features = ["Pclass", "Sex", "Age", "SibSp", "Parch", "Fare",
"Embarked"]
X = titanic[features] y
= titanic["Survived"]
 print("\nКоличество пропусков в выбранных
признаках:") print(X.isnull().sum())
```

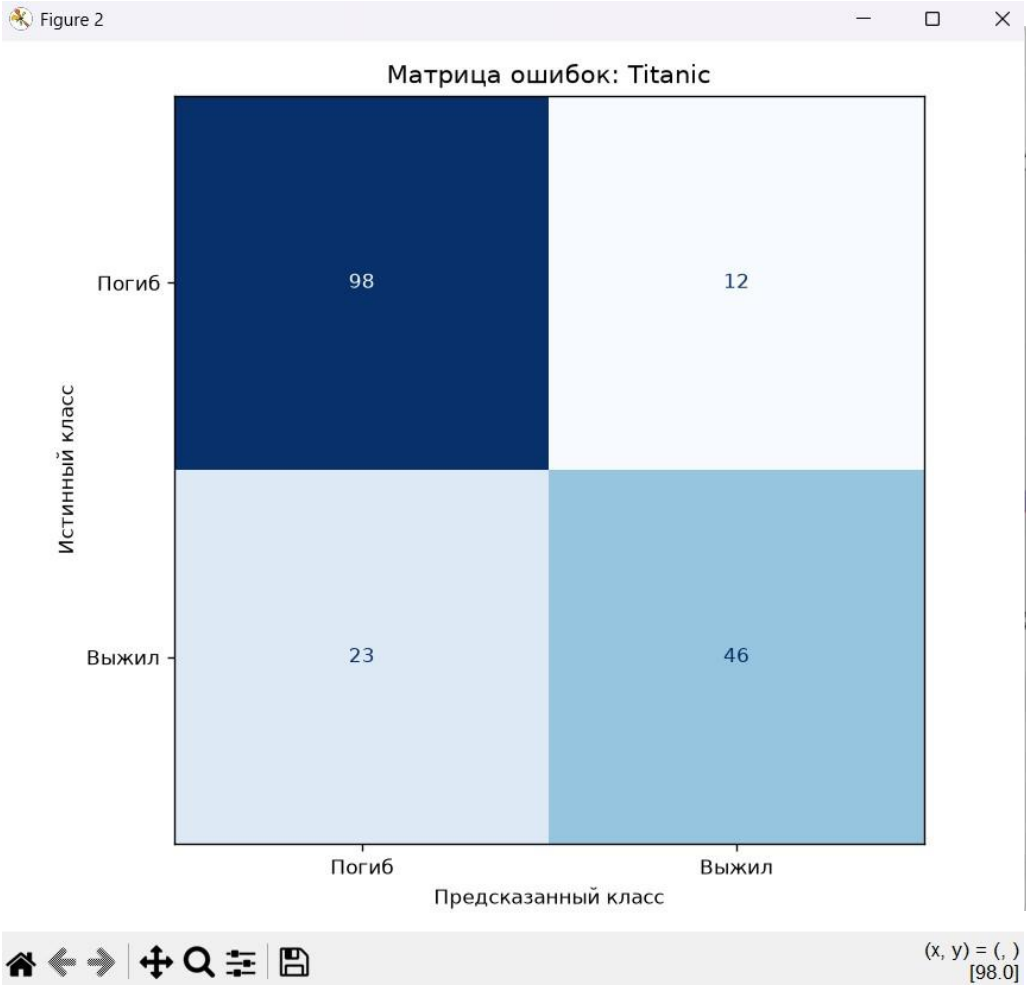
```
X_train, X_test, y_train, y_test = train_test_split(
X,     y,     test_size=0.2,     random_state=42,
stratify=y
)  numeric_columns = ["Age", "SibSp", "Parch",
"Fare"] categorical_columns = ["Pclass", "Sex",
"Embarked"]
 numeric_transformer =
Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])  categorical_transformer =
Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])  preprocessor =
ColumnTransformer([
    ("numeric", numeric_transformer, numeric_columns),
("categorical", categorical_transformer, categorical_columns)
])  logistic_model = Pipeline([
("preprocessor", preprocessor),
    ("model", LogisticRegression(max_iter=1000, random_state=42))
])  logistic_model.fit(X_train,
y_train) y_pred =
logistic_model.predict(X_test)
 print(f"\nОбучающая выборка: {len(X_train)}") print(f"Тестовая выборка:
{len(X_test)}") print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print(f"Precision: {precision_score(y_test, y_pred, zero_division=0):.4f}")
print(f"Recall: {recall_score(y_test, y_pred, zero_division=0):.4f}")
 results =
pd.DataFrame({
    "Реальный класс": y_test.to_numpy(),
    "Прогноз": y_pred
})  print("\nПримеры предсказаний (0 — погиб, 1 —
выжил):")
print(results.head(10).to_string(index=False))
 matrix = confusion_matrix(y_test, y_pred, labels=[0,
1]) tn, fp, fn, tp = matrix.ravel()
 print("\nМатрица
ошибок:") print(matrix)
 print("\nАнализ матрицы ошибок:") print(f"Верно
предсказана гибель пассажира: {tn}") print(f"Ошибочно
предсказано выживание: {fp}") print(f"Ошибочно
предсказана гибель: {fn}") print(f"Верно предсказано
выживание пассажира: {tp}")
 fig, ax = plt.subplots(figsize=(7,
6)) display = ConfusionMatrixDisplay(
confusion_matrix=matrix,
display_labels=["Погиб", "Выжил"]
) display.plot(ax=ax, cmap="Blues",
colorbar=False) ax.set_title("Матрица ошибок:
Titanic") ax.set_xlabel("Предсказанный класс")
```

```
ax.set_ylabel("Истинный класс")
plt.tight_layout()
 plt.show()
```

Результат:



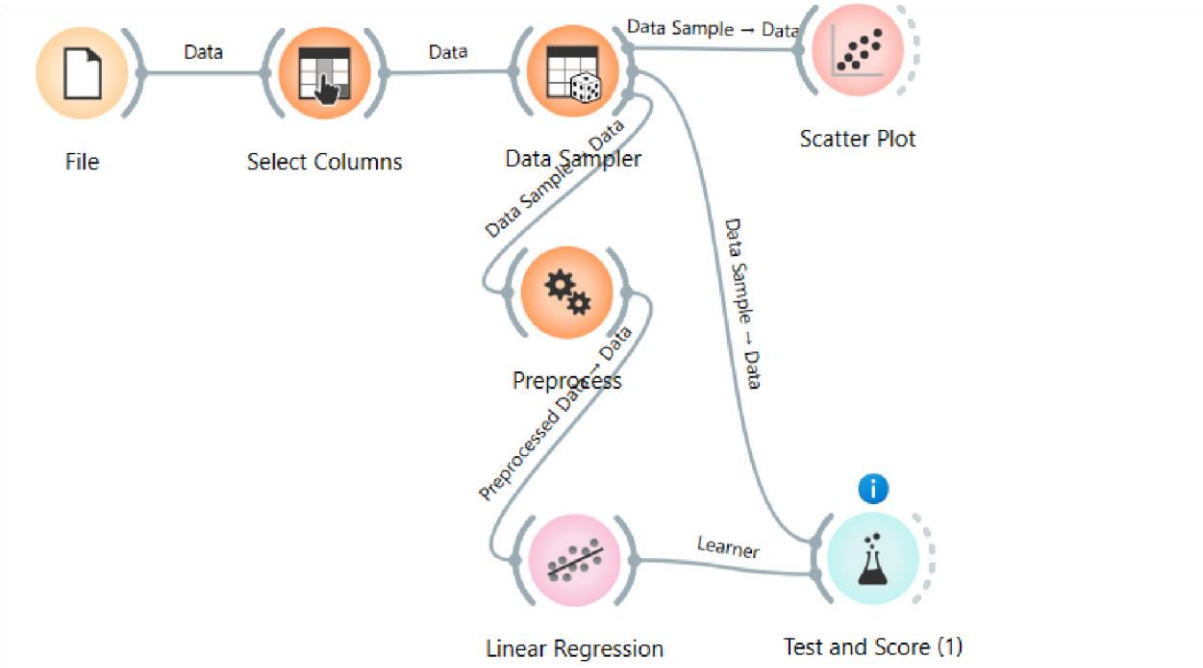
Запуск задачи регрессии (California Housing)
MSE: 4908290571.35
R^2: 0.6254

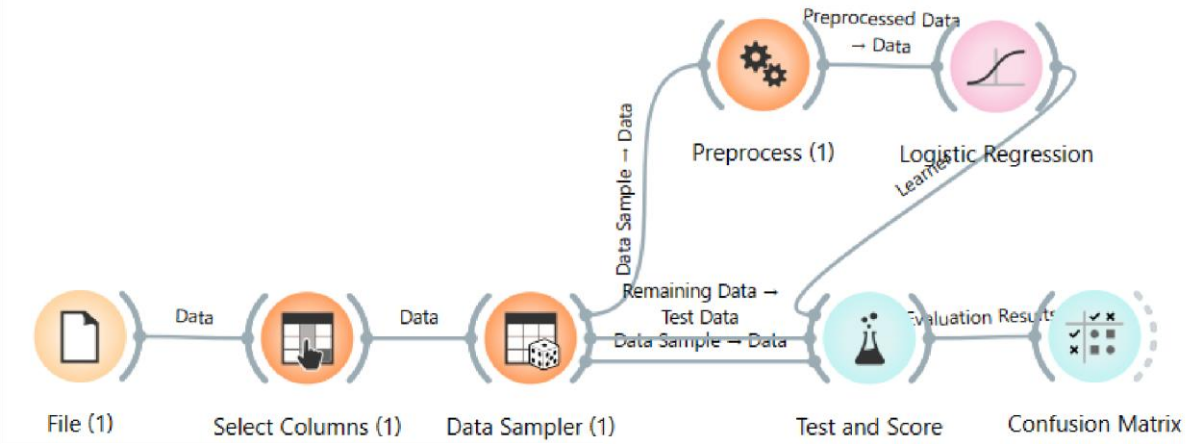


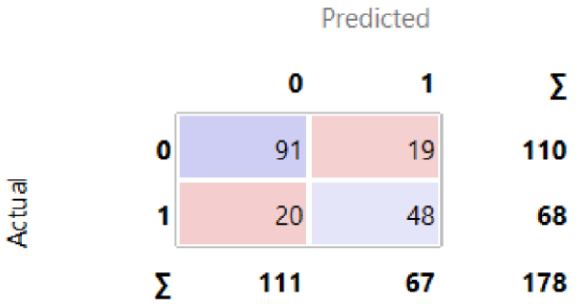
Запуск задачи классификации
Accuracy: 0.8045
Precision: 0.7931
Recall: 0.6667

Анализ матрицы ошибок:
Верно предсказана гибель пассажира: 98
Ошибочно предсказано выживание: 12
Ошибочно предсказана гибель: 23
Верно предсказано выживание пассажира: 46

Результаты в Orange:







Вывод: в ходе выполнения лабораторной работы были успешно освоены методы линейной и логистической регрессии для решения практических задач. Были обучены модели для задач регрессии и классификации, рассчитаны ключевые метрики их качества и проанализированы полученные результаты.